

COMP3438 System Programming

Overview of Unix

Course Organization

Part I: Unix System Programming (Device Driver Development)

Character Device Driver
Development

Introduction to Block
Device Driver

Overview of Device Driver Development

Process

File System

Overview of Unix Systems Programming

Part II: Compiler Design

Syntax Analysis

Lexical Analysis

Overview of Compiler Design

Overview of the Subject

Unix

"Unix is a family of multitasking, multiuser computer operating systems that derive from the original AT&T Unix, development starting in the 1970s at the Bell Labs research center by Ken Thompson, Dennis Ritchie, and others." -- Wikipedia

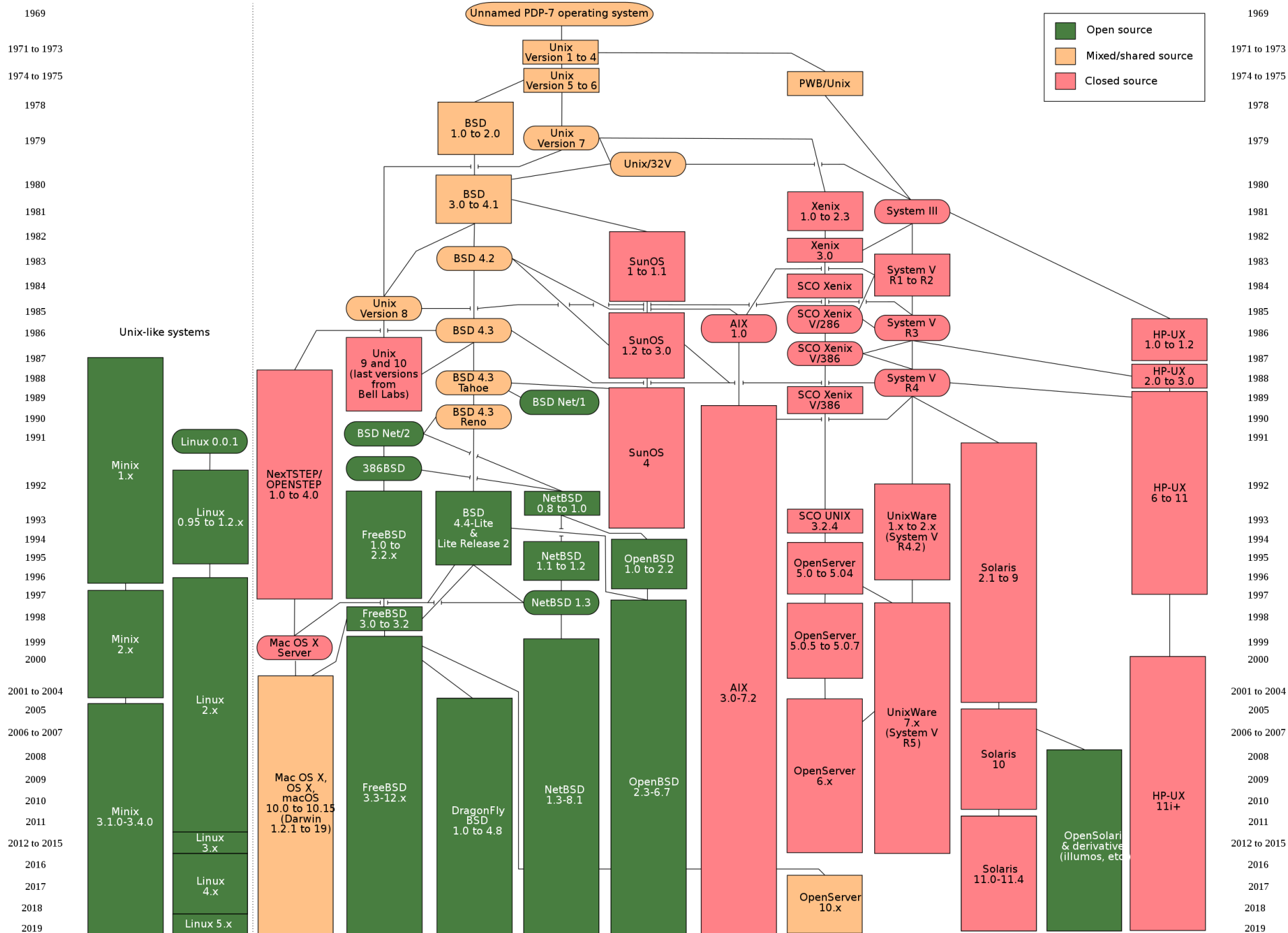
Unix

Unix family tree, quoted from Wikipedia

Originally written in
assembly

In 1973, v4
was mostly rewritten
in C, making it
portable

Portable Operating System Interface (POSIX) to unify Unix APIs



The Unix Philosophy

Modular design

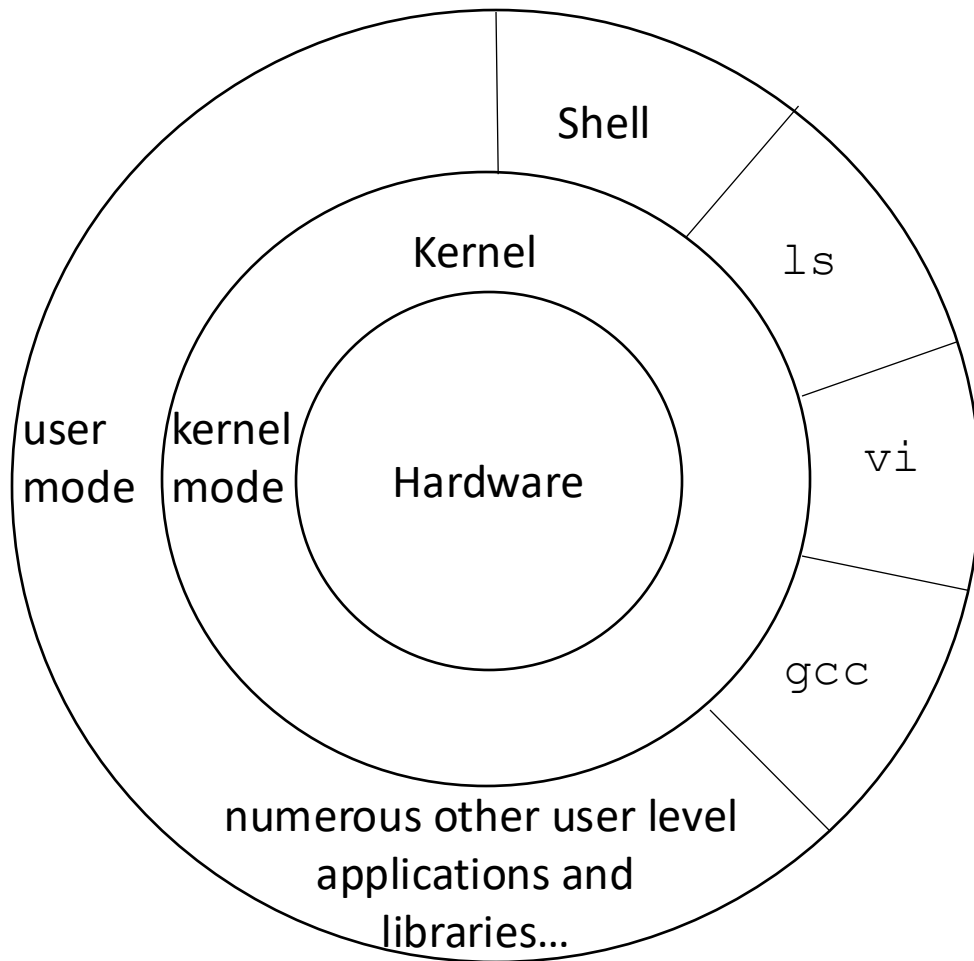
A collection of simple tools, with limited well-defined functions

File system

Inter-process communication

The tools can be easily composed (via `>`, `>>`, `<`, `|`, `` ``) by a shell to accomplish complex tasks

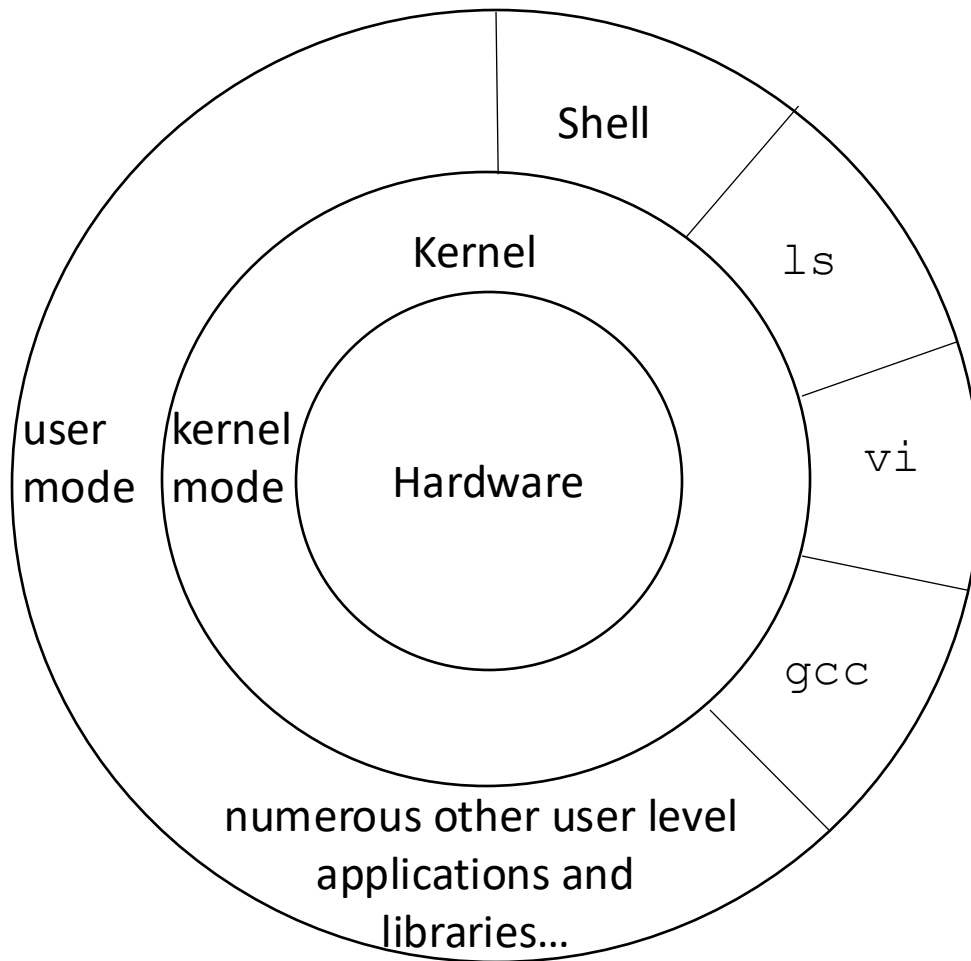
What makes up Unix



Kernel is the heart of the OS. It interacts with the hardware, and conduct key functions of the OS, such as memory management, task scheduling, file management

Shell is the utility that interacts with the user via command line. When you type a line command from your terminal input device, the shell interprets the command and execute the intended program (or composition of programs). The shell commands have a standard syntax. C Shell, Bourne Shell, and Korn Shell are the most famous shells, which are available in most Unix variations (Linux commonly use the Bourne Shell – like `bash`).

What makes up Unix

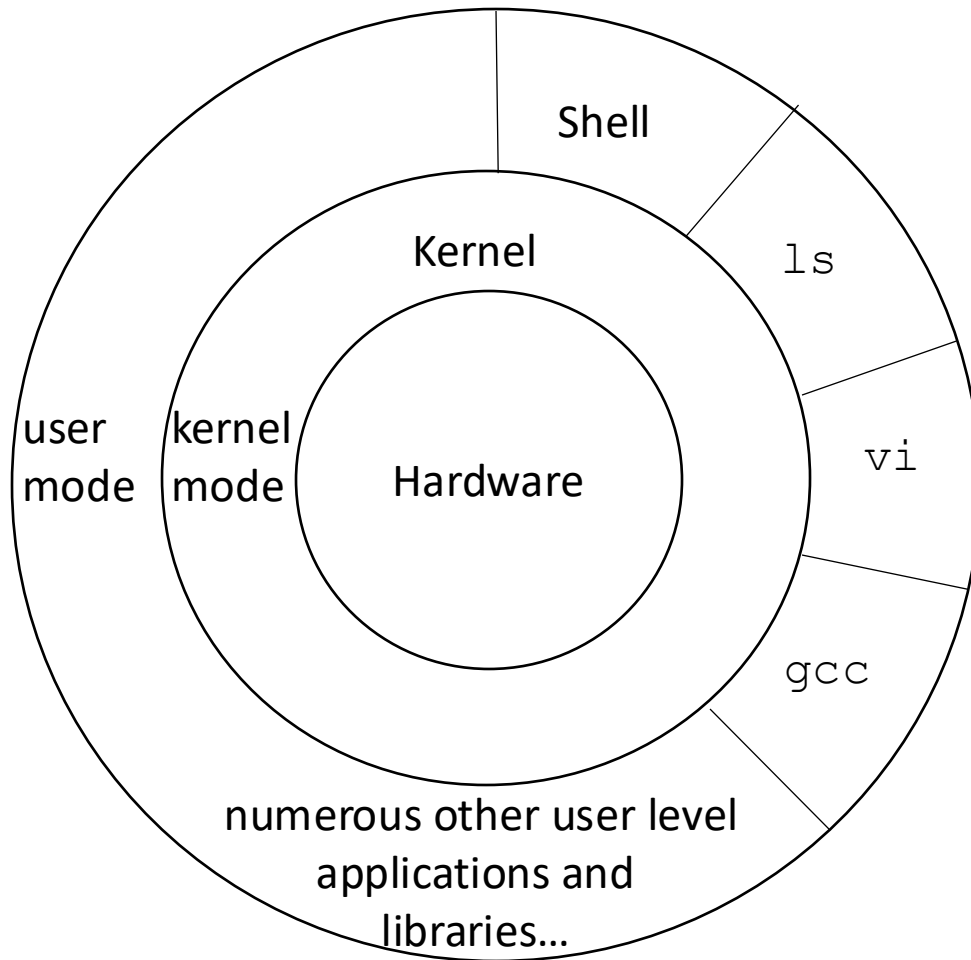


Commands and utilities support various day to day activities, e.g. `ls`, `vi`, `gcc`, `cp`, `mv`, `cat`, `grep`. There are over 250 standard commands and utilities plus numerous other 3rd party software. Most commands and utilities come with various options. These build up a huge vocabulary to express all kinds of needs for users.

Files and directories organize data. All files are organized into directories, and directories are further organized into a tree-like structure called the file system.

Demo Task1-9

Unix Kernel



The kernel is the core of the Unix OS

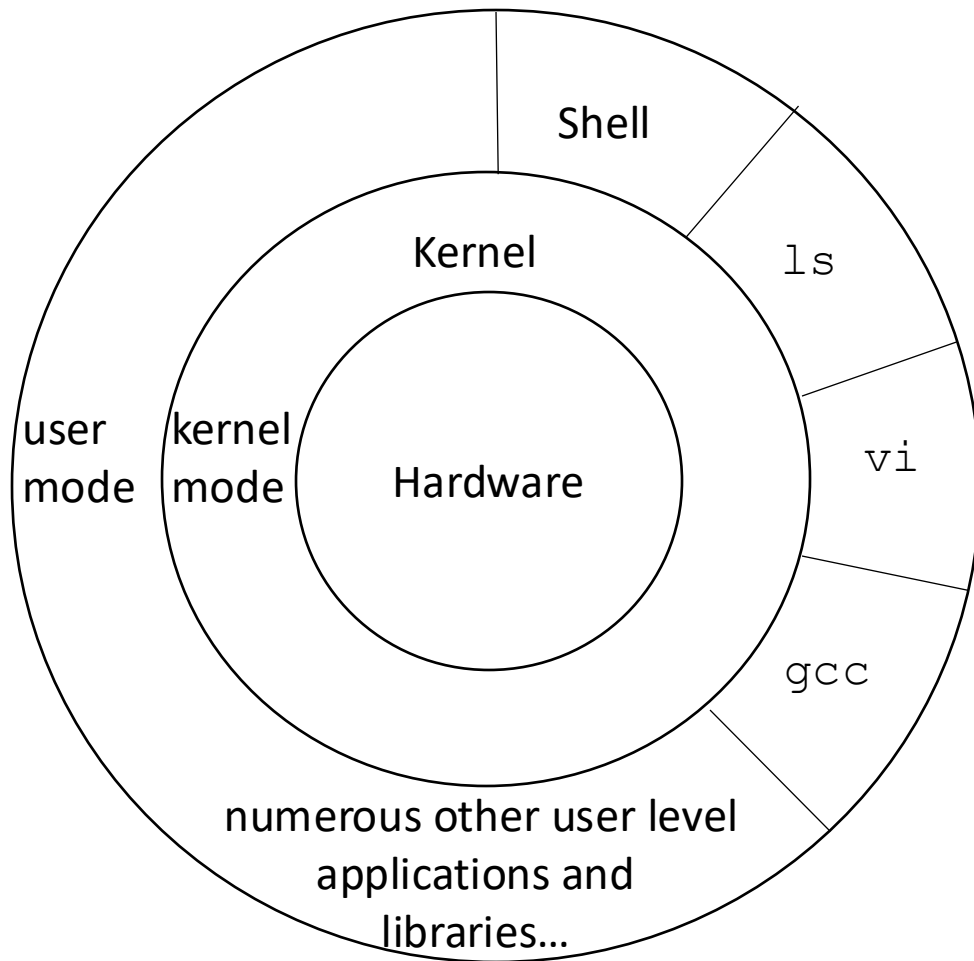
Controls the hardware and performs various low-level functions

A collection of software modules that runs in the privileged mode (full access to system resources)

Main duties:

- **process** scheduling and management,
- **inter-process** communication,
- **memory** management,
- **file** management,
- **network** stack management,
- date and **time** services,
- system **accounting**,
- **security**,
- **device** management,
- **interrupt** and **exception** handling

Unix Kernel and System Calls



Other parts of the Unix system, as well as user programs, request the kernel for various services through **system calls**

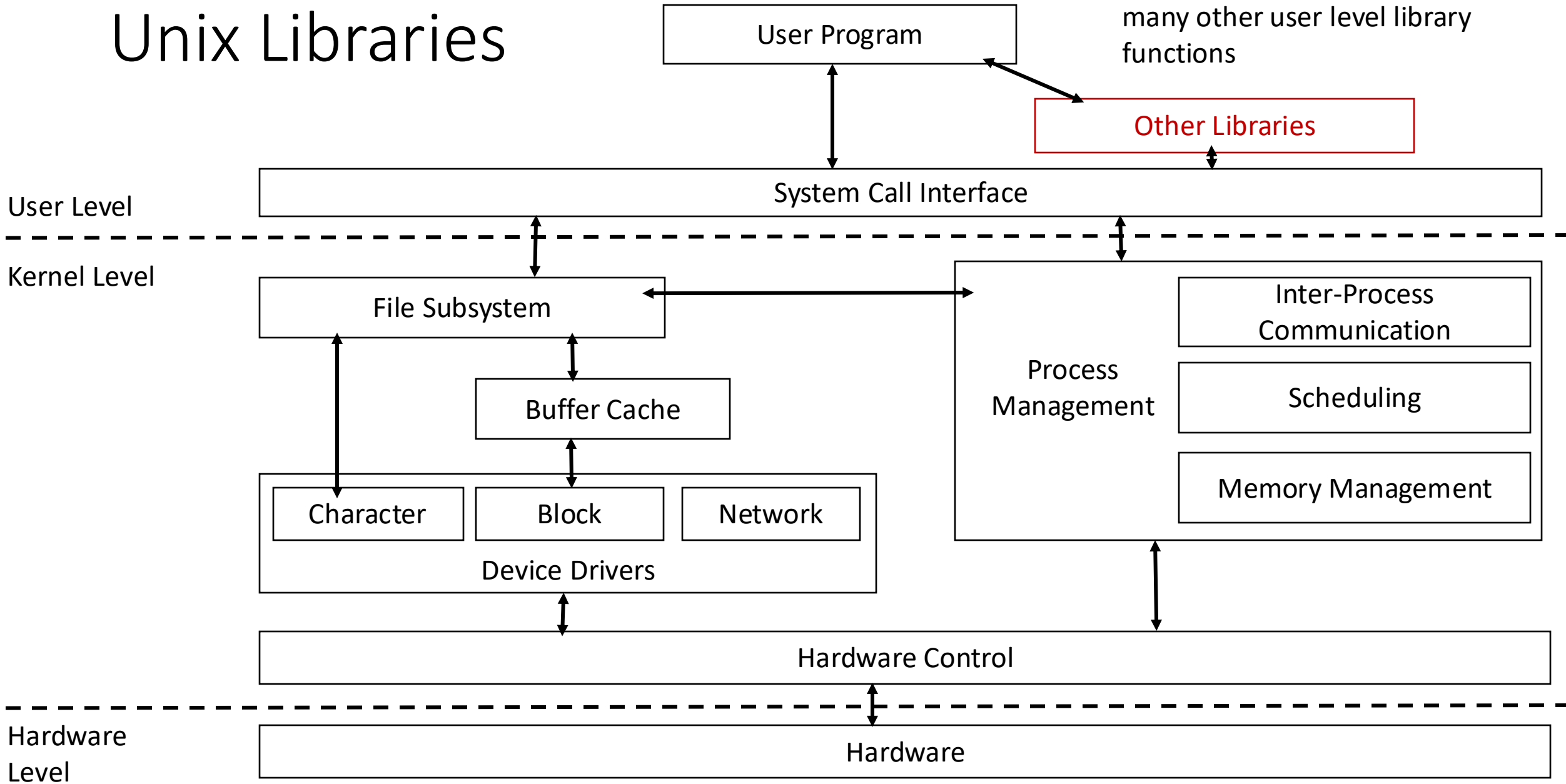
A system call is an entry point into the kernel, typically packaged as a function call, as part of the OS API.

Inside of the function call, the typical implementation:

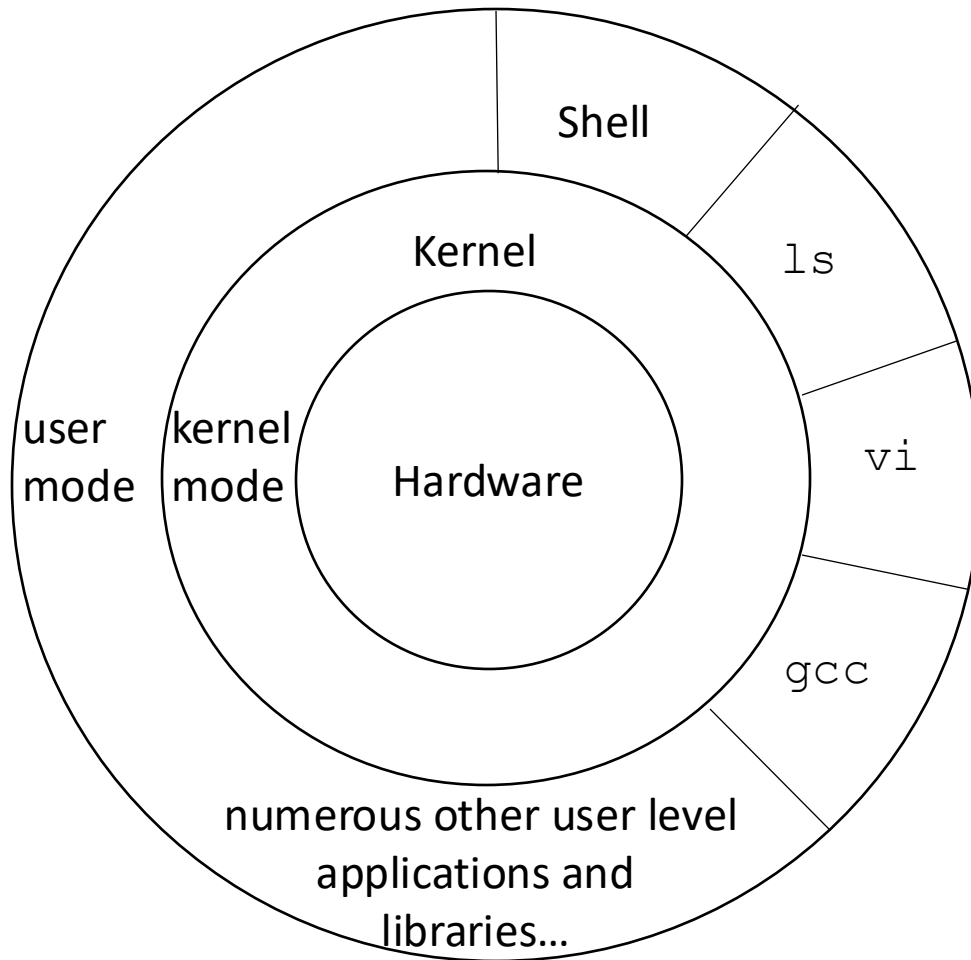
1. a software interrupt (trap) switches the CPU hardware to the **kernel mode**
2. execute kernel routines
3. switches back (via scheduling*) to the **user mode** to resume the user application.

Unix Libraries

Besides the system call libraries, user programs can also call the many other user level library functions



Unix Commands and Utilities



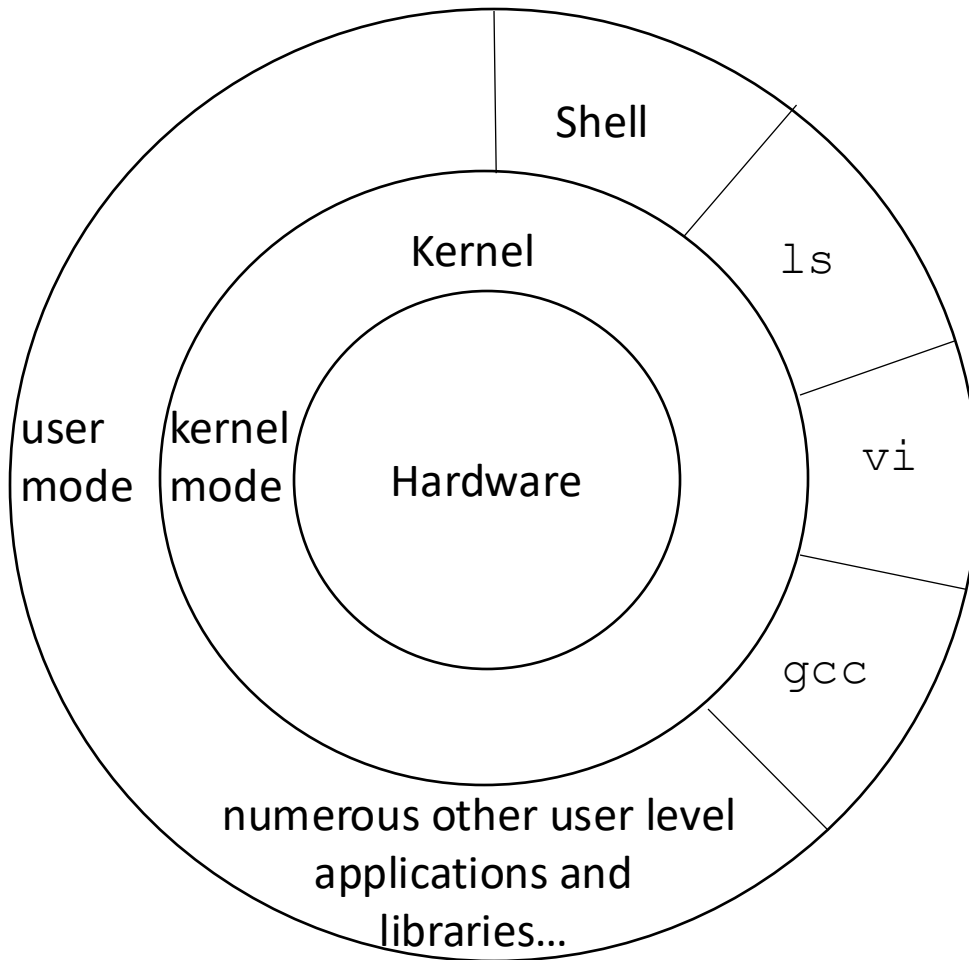
A large set of tools for various basic user level functions

A vocabulary and a grammar, to combine basic user level functions to nearly arbitrary sophisticated functions

Not part of the kernel, but a part of the OS

Typically accessed via the Shell

Unix Shell



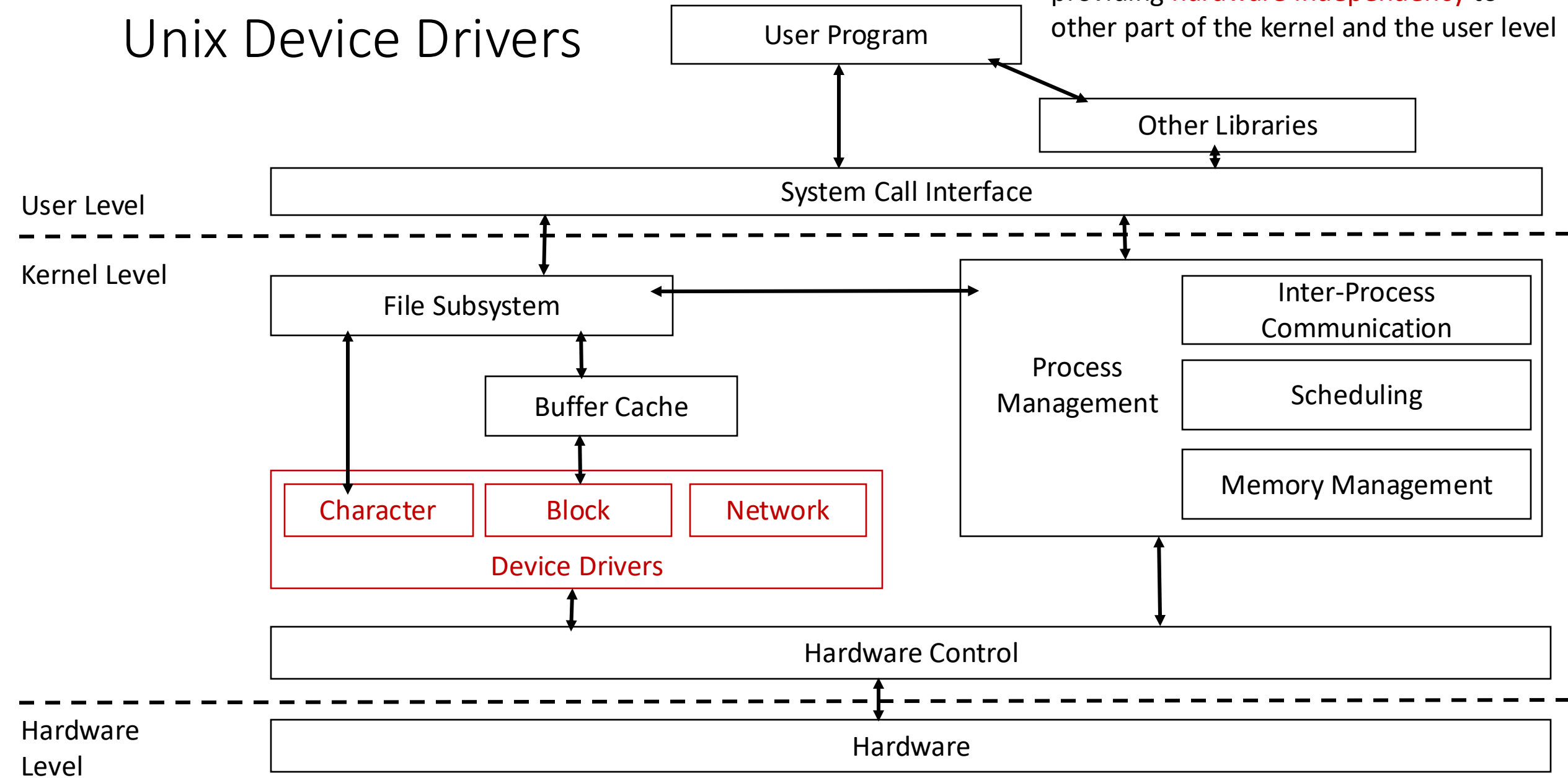
A powerful command interpreter (a user level application) for Unix – accepts user text commands and carries them out

Can combine various user level applications to serve more sophisticated functionalities

Multiple shells for a same user, or multiple shells for multiple users can co-exist

Unix Device Drivers

Device drivers **encapsulate** the hardware, providing **hardware independency** to other part of the kernel and the user level



Features of Unix

- Portability
- Multiuser Operation
- Multitask Processing
- Hierarchical File System
- Powerful Shell
- Pipes
- Networking
- Robustness

Unix is Portable

Unix is a relatively hardware independent OS. Various mechanisms (device driver, various C program interfaces inside of the kernel and to the user level) are designed to encapsulate the hardware specifics, facilitating porting between hardware platforms.

One key to the portability is the device drivers, specific modules that encapsulate the hardware details from the other parts of the kernel and the user level.

Unix supports multi-users

Unix is a multi-user, multi-tasking OS

Multiple users may run multiple tasks concurrently. This is very different from conventional PC OSs, such as MS-DOS or Windows, which allows concurrent execution of multiple tasks, but not multiple users.

Unix supports multi-tasking

Unix is a multi-tasking OS

Even for a single user, time sharing can still support multi-tasking.

Unix has some programs, such as the system-wide accounting programs, that automatically run from time to time.

Unix supports background processing, which allows a user to initiate a task "in the background" and then proceed to other activities. Unix time shares between the front-end commands and background jobs.

Unix allows the creation of new tasks from an existing task.

Unix has a hierarchical file system

Unix files are organized into separate directories.

Directories are themselves organized into a tree-like structure. There is one master directory, the so-called root directory, from which various sub-directories branch off.

The hierarchical structure offers a maximum flexibility for grouping information in a way that reflects its natural structure.

- A single user's data may be grouped by activity
- Data from many different users can be grouped according to corporate organization
- As a result, stored data is easier to locate and manage

Unix shell is powerful

The Unix shell supports a number of convenient features, such as:

Redirection of application input and output, e.g.

```
ls > myfiles
```

The ability to manipulate groups of files with a single command.

Executing a sequence of commands stored in a text file, called a "shell script," allowing us to build our own commands, which may be parameterized.

Being used as a programming language that provides string-valued variables and control flow primitives including branching and iteration

Demo Task10-13

Unix's pipe is novel

One of the most famous Unix features is pipe.

A pipe passes the standard output of one command directly to another command, to be used as its standard input, e.g.

```
who | sort
```

Allows any number of commands to be connected in a sequence, and automatically handles the data flow from one program to the next

```
who | sort | lp
```

Produces the same effect as if one large program, rather than several small ones, had been executed.

- Allowing the combination of several simple programs to perform more complex functions
- Eliminating the need for new software development.

Unix supports networking

Networking support is built into the Unix system.

Supports TCP/IP protocols, and provides a new OS abstraction, the **socket**, that allows application-level programs to access the Internet.

The socket abstraction acts as an interface between application level programs and the underlying TCP/IP protocols.

Unix is robust

When encountered an error, a Unix program does not abort. Instead, the program receives a returned value indicating an error condition, and it is up to the program to check for the error and handle it.

Typically, a returned error value is negative if the return type is `int`, or a `NULL` if the return type is a pointer.

You can call the C library function `perror()` to output a message string to the standard error file, to further explain the error.

Unix is robust

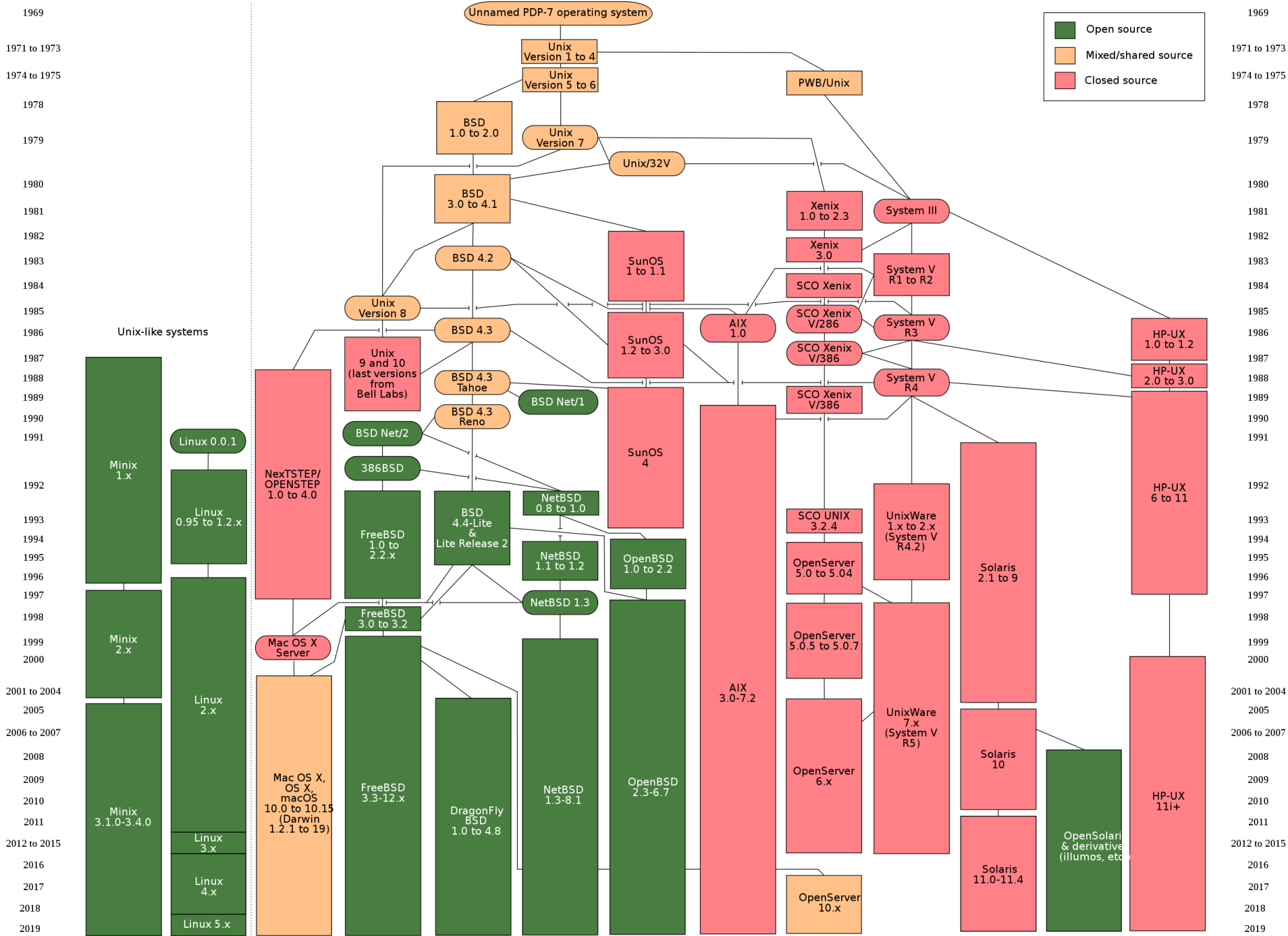
Example: the use of the open system call.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int fd;
if ((fd = open("my.file", O_RDONLY)) == -1)
    perror("Unsuccessful open of my.file");
```

The following output might be produced if `my.file` does not exist:

```
Unsuccessful open of my.file: No such file or directory
```

Unix Variants



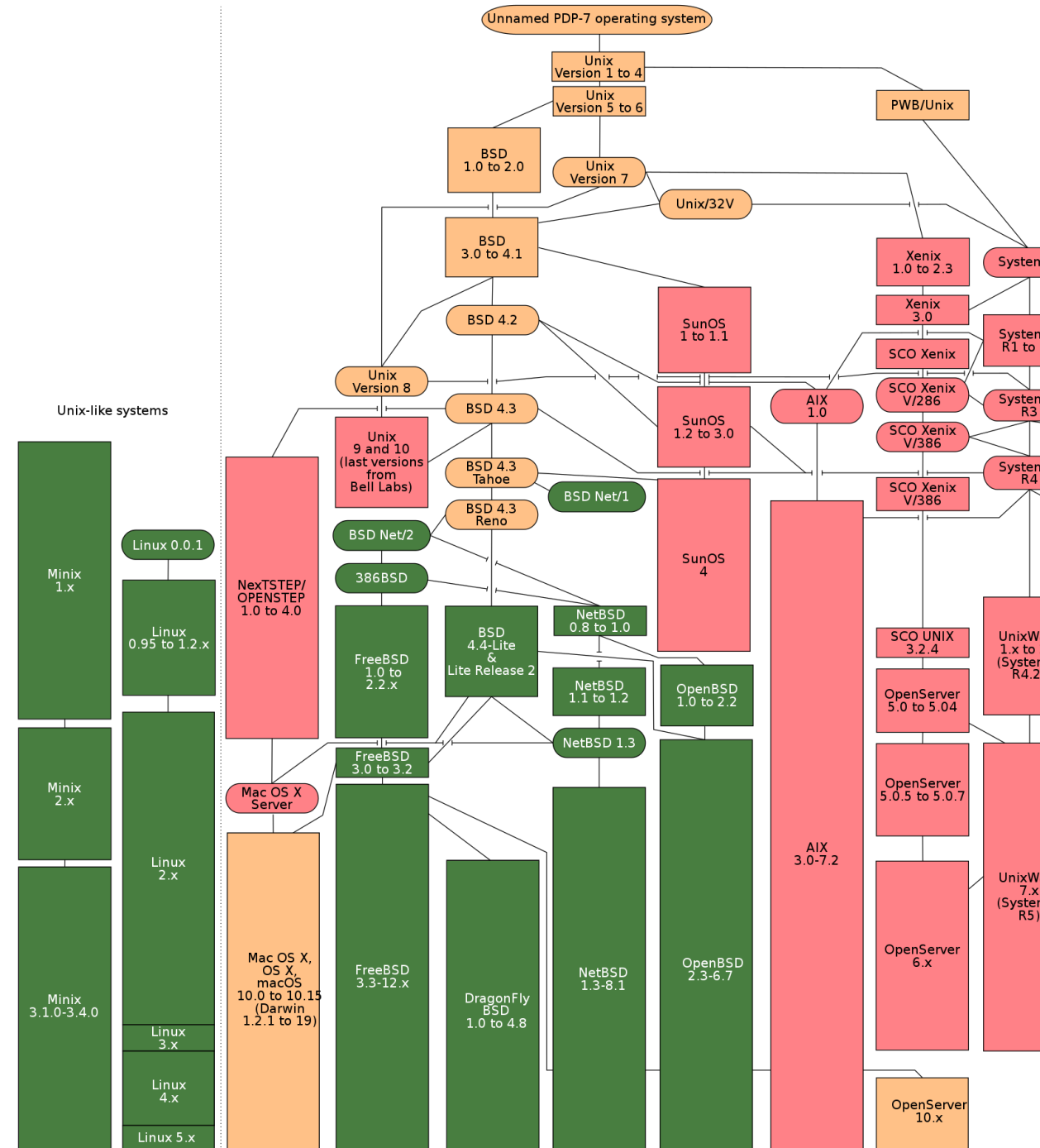
Linux

Strictly speaking, Linux source code is not inherited from the Unix family tree. Linux is a Unix-like OS.

But if you consider Portable Operating System Interface (POSIX) standards, then Linux can be regarded as a kind of Unix

"Linux is a Unix clone written from scratch by Linux Torvalds with assistance from hackers across the Net" -- official Linux kernel README file

1969
1971 to 1973
1974 to 1975
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001 to 2004
2005
2006 to 2007
2008
2009
2010
2011
2012 to 2015
2016
2017
2018
2019



Android (based on Linux)

Android architecture
(quoted from Wiki)

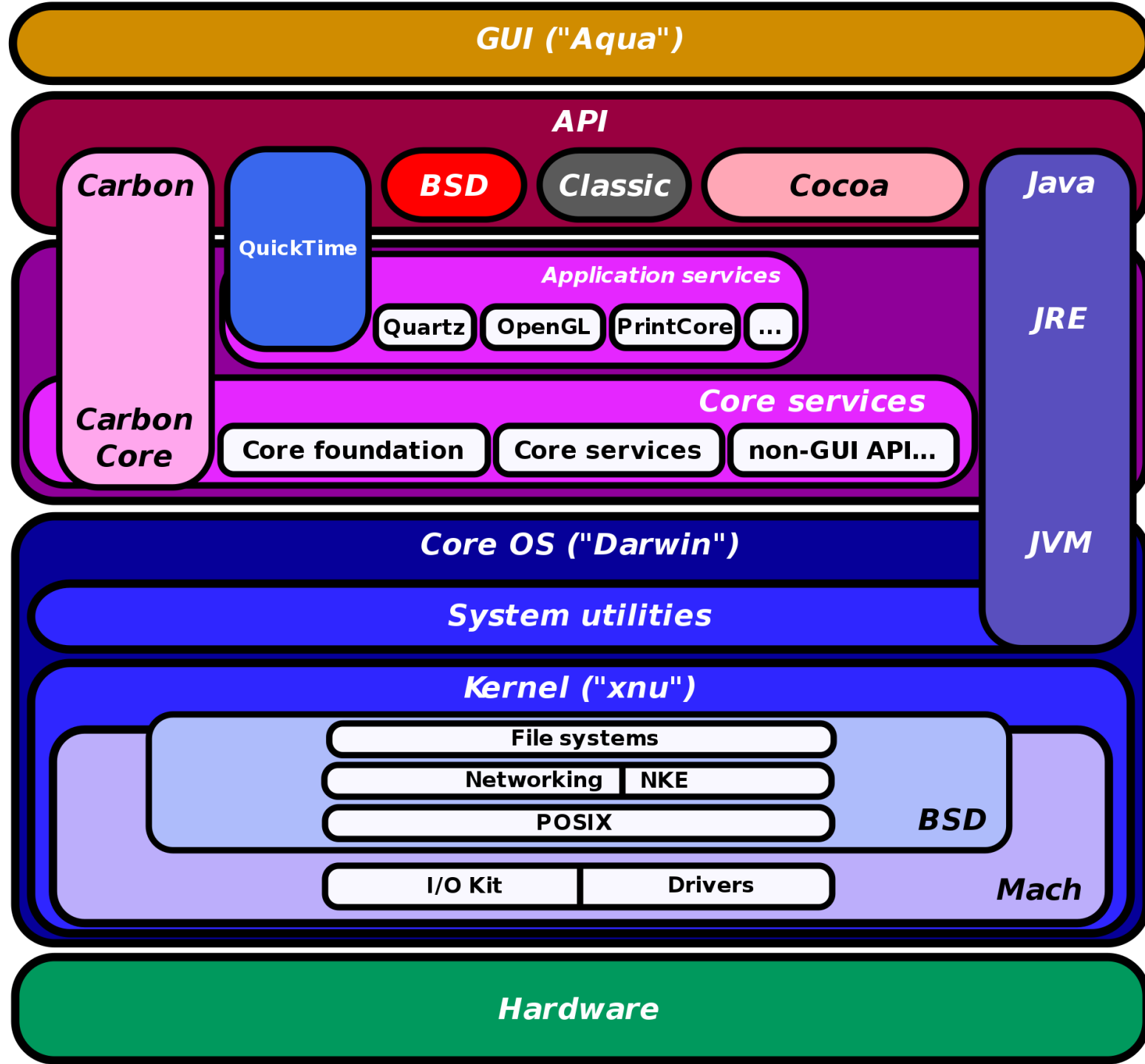


A nice website for beginners:

<http://www.ece.ncsu.edu/wireless/MadeInWALAN/AndroidTutorial/index.html>

Mac OS X

Mac OS X architecture
(quoted from Wiki)



Links to other Unix resources

For Unix history and standardization, see

<http://www.UNIX-systems.org/>

For people and stories about Unix and its descendants, see

<http://www.bell-labs.com/history/unix>